

20. Arrays and pointers, and reserving memory on the fly

Herman Kamper

This lecture does two things. The first is to explain how pointers and arrays relate to one another, and this can be quickly summarised: arrays are pointers. There are some notational differences, but if you understand pointers and you understand arrays, you will see that the two are linked and that you can use one wherever you expected the other.

The second thing is to show you how to reserve memory on the fly. Up to now, every program we have written has been one where the compiler could look at what you wrote and work out almost exactly how much memory it would need for all your variables. Real programs are not like that. We want to be able to block off regions of memory while the program is running, as and when we need them.

Arrays are pointers

Start with the easy one. Suppose you write this:

```
int a[4];  
int *myPtr;
```

The first line creates four boxes in the computer's memory: `a[0]`, `a[1]`, `a[2]` and `a[3]`. Each of those boxes has an address. Say the first one sits at address 3000. Each box holds an `int`, so each box is 4 bytes (32 bits). Crucially, an array's boxes sit next to each other in memory. So `a[1]` sits at 3004, `a[2]` at 3008 and `a[3]` at 3012. If you just write those two lines and nothing else, the boxes contain rubbish left over from whatever was there before.

If you write `a` on its own, without a subscript, what do you get? Up to now you have probably thought of `a` as referring to all four boxes together. That is not really true. In almost every expression you can write, the name of an array turns into the address of its first element. You can think of `a` on its own as being 3000.

The second line above creates one more box, somewhere else in memory, called `myPtr`. That box is exactly the size needed to store an address, and at the moment it contains rubbish too. Now we do:

```
myPtr = &a[0];
```

This puts 3000 into the `myPtr` box. Nothing surprising.

```

int a[4];
int *myPtr;
myPtr = &a[0];
myPtr++;
myPtr = myPtr + 2;

```

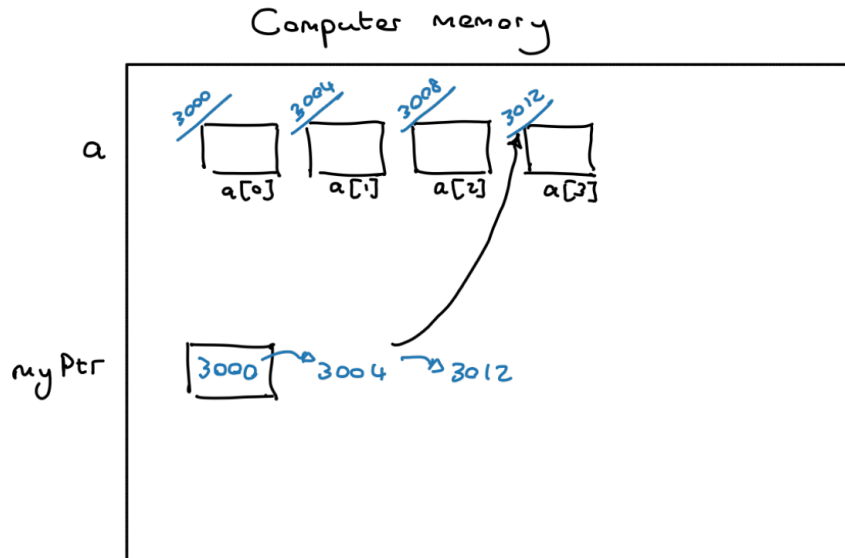


Figure 1: The array `a` and the pointer `myPtr` in the computer's memory. Each of the four boxes of the array holds an `int`, so they sit 4 bytes apart at 3000, 3004, 3008 and 3012.

Now let us add the next line, which causes the first surprising thing:

```
myPtr++;
```

What do you guess that does? A reasonable guess is that it increments the value in the box that `myPtr` points at. That is wrong, but it is a reasonable guess: if you wanted that, you would write `(*myPtr)++`. What `myPtr++` actually does is make the pointer point at the next box in the array (address 3004 in the diagram).

Be precise about why. The number inside the `myPtr` box increases by however many bytes it takes to store the type of thing being pointed at. `myPtr` is an `int *`, an `int` takes 4 bytes, so the value in the box goes from 3000 to 3004. And 3004 happens to be exactly where `a[1]` starts. Adding one to a pointer means “move on by one of the things I point at”, not “add one to the address”.

The same is true for larger steps, as in the next line:

```
myPtr = myPtr + 2;
```

Plus two does not mean plus two. It means two of whatever size the pointed-at type is, so 3004 becomes 3012, because we are adding 2×4 bytes. The result is that we are pointing at `a[3]`, the last box of the array sitting at address 3012.

You can now see why arrays and pointers are tightly linked: stepping a pointer along is exactly the same motion as stepping an index along.

One warning while we are here. Pointers do not check anything. If you increment `myPtr` once more, it moves past the end of the array and points at whatever happens to sit in memory after `a`. C will not stop you and will not give you an error. Pointers do what you tell them to do.

Four ways to reach the same box

```
/*
 * Illustrate the relationship between pointers and arrays.
 */

#include <stdio.h>

int main()
{
    int a[4] = {99, 1, 3, 26};
    int *myPtr;

    myPtr = &a[0];

    printf("Value in array: %d\n", a[2]);
    printf("Value via Ptr : %d\n", *(myPtr + 2));
    printf("Value via Ptr : %d\n", myPtr[2]);
    printf("Value via Ptr : %d\n", *(a + 2));

    return 0;
}
```

All four printf lines print the same thing:

```
Value in array: 3
Value via Ptr : 3
Value via Ptr : 3
Value via Ptr : 3
```

Take them one at a time, because each is a different notation for the same operation.

`a[2]` is ordinary array subscript notation, which you have been using since arrays were introduced. It gives 3.

`*(myPtr + 2)` is pointer/offset notation. The `myPtr + 2` part moves on two int-sized blocks from wherever `myPtr` currently points, and the `*` then opens that box and gets the value out. Note that `myPtr` itself does not move: `myPtr + 2` is an expression, not an assignment.

`myPtr[2]` is the strange one. `myPtr` is not an array. It is just a box with an address in it. But C sees you doing something array-like and does exactly what the square brackets always do: move on that many positions, then dereference. So `myPtr[2]` and `*(myPtr + 2)` are the same thing written two ways. This is called pointer subscript notation.

`*(a + 2)` closes the circle. Here `a` is an array, but we said the name of an array turns into the address of its first element, so `a + 2` is pointer arithmetic on that address and the `*` dereferences it. Array subscript notation was pointer arithmetic all along: `a[2]` is *defined* to mean `*(a + 2)`.

Because `a` is already an address, you do not even need the `&a[0]`. This works just as well, and is

the form you will usually see:

```
int *myPtr = a;
```

It is worth being clear about which of these move the pointer and which do not. `*(myPtr + 2)` and `myPtr[2]` leave `myPtr` exactly where it was: they compute an address, use it and throw it away. Only an assignment like `myPtr++` or `myPtr += 2` actually changes what is in the `myPtr` box. Here is a program that does move it:

```
int a[4] = {99, 1, 3, 26};
int *myPtr = a;

printf("myPtr      : %p\n", myPtr);
myPtr++;
printf("myPtr      : %p\n", myPtr);
myPtr += 2;
printf("myPtr      : %p\n", myPtr);
printf("*myPtr      : %d\n", *myPtr);
```

which prints something like

```
myPtr      : 0x7fff56c96800
myPtr      : 0x7fff56c96804
myPtr      : 0x7fff56c9680c
*myPtr      : 26
```

The exact addresses will differ every time you run it, but the pattern will not. The address ends in 00, then 04 after one step, then 0c after two more. Those are hexadecimal, so 04 plus 8 is 0c. Three steps of 4 bytes from the start is `a[3]`, which holds 26.

This is the sort of place where pointer notation earns its keep. If you are looping through an array, you can keep a pointer and walk it forward instead of indexing, and for anything intricate that is often easier to write than the subscript version.

Pointer expressions

Arithmetic is not the only thing you can do with pointers. The diagram below shows a five-element `int` array on a machine with 4-byte ints, with a pointer aimed at the first element.

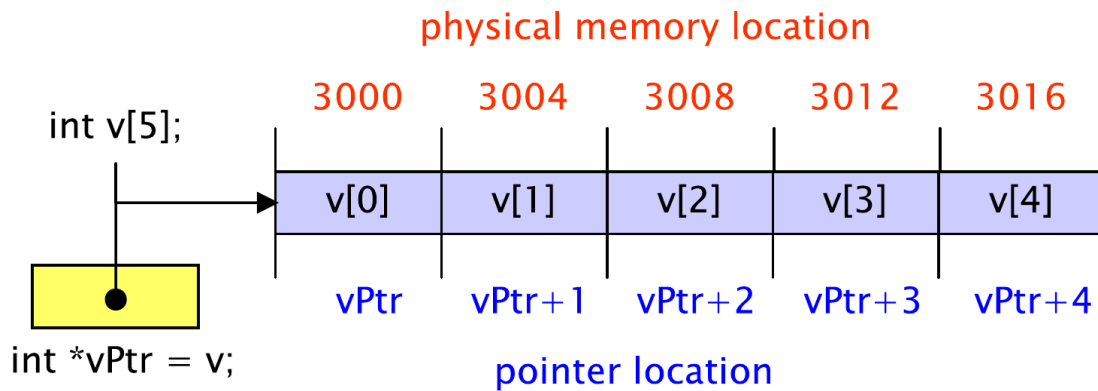


Figure 2: A five-element `int` array with `vPtr` pointing at the first element. The addresses along the top go up in steps of 4 because an `int` takes 4 bytes, and the labels underneath show which element each offset from `vPtr` reaches.

The first thing you can do is subtract one pointer from another:

```
int v[5], *vPtr0, *vPtr2;
```

```
vPtr0 = &v[0];
```

```
vPtr2 = &v[2];
```

Now `vPtr2 - vPtr0` produces 2. Note carefully what that 2 means. It is not the difference between the two addresses in bytes, which would be 8. Pointer subtraction gives you the number of elements from one to the other. It is asking “how many ints would I have to step to get from here to there?”

The second thing you can do is compare pointers, with `<`, `>` and `==`. So `(vPtr0 + 1) > vPtr0` is true, because `vPtr0 + 1` points at a later element. Asking which of two pointers sits earlier in memory is unusual; you might want it if you were writing something very low-level and needed to check that a pointer had stayed inside a particular block of memory. Comparing for equality is much more common: are these two pointers looking at the same thing? Remember that this is all it is doing, comparing the number inside one pointer box with the number inside another.

The comparison you will write most often is against `NULL`:

```
if (vPtr == NULL)
{
    // vPtr is not pointing at anything usable
}
```

`NULL` is the conventional way of saying that a pointer points at nothing.

All of these operations only make sense on a pointer into an array. Subtracting two pointers that point at two unrelated variables tells you nothing useful.

Why a pointer has a type

Now a question that trips many students up. I am going to say something that sounds controversial: pointers do not really point at anything.

A pointer is a box with a number inside it, and that number happens to be an address. That is all it is. Whether it is a `char *`, an `int *` or a `double *`, what is inside the box is an address. The arrows we draw in these diagrams do not exist anywhere in the machine; they are a reminder to ourselves about what is living at the address inside the box.

Which raises a fair objection. If the box just holds an address, why do we have to write a type in front of the `*` at all? What is the `int` in `int *myPtr` actually doing? It does not change the box and it does not change the address.

The answer is that it tells C what to do when you dereference. The whole argument is in the sketch below. The rest of this section walks through it line by line.

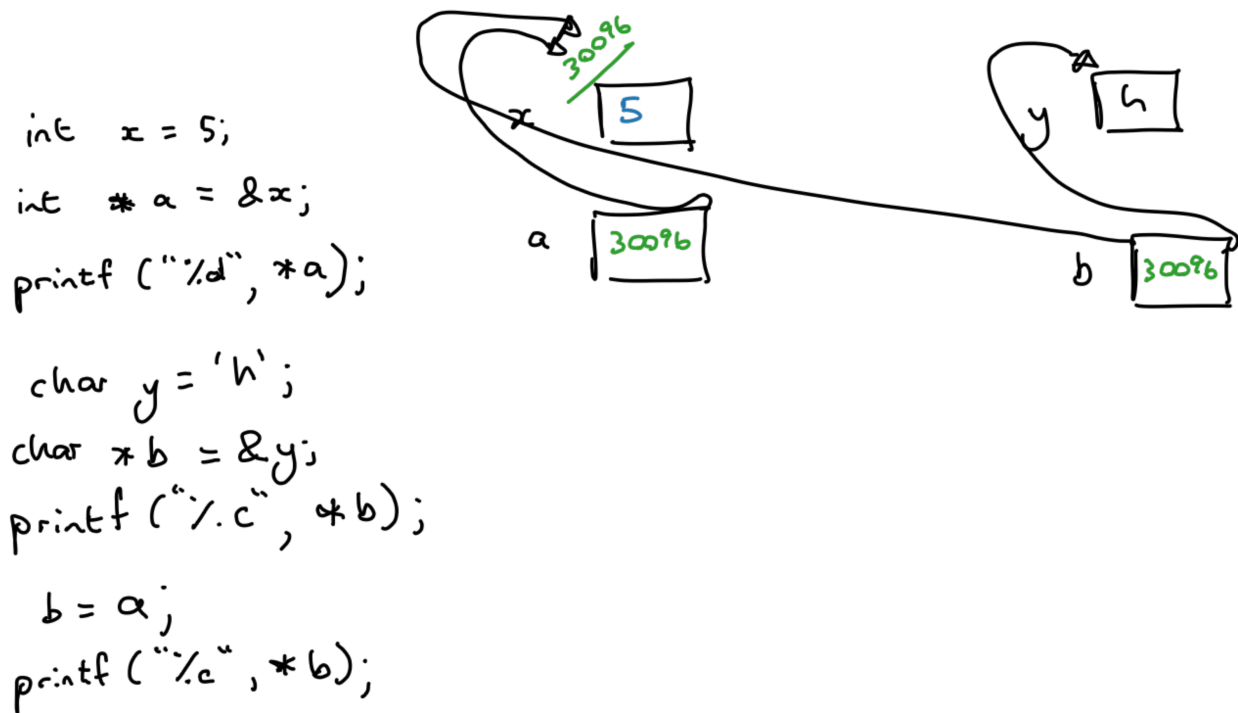


Figure 3: Two pointers aimed at the same address. `a` is an `int *` and `b` is a `char *`, so after `b = a` both boxes contain 30096, but dereferencing `a` reads 32 bits from there while dereferencing `b` reads only 8.

```
int x = 5;
int *a = &x;
printf("%d\n", *a);
```

`x` is a box holding 5. Say it lives at address 30096. Then `a` is a new box, and what goes in it is 30096. I know an address goes in it because of the `*`; the `int` has not done anything yet. The `printf` prints 5. But look at *how* it prints 5. It goes to address 30096, and then, because `a` is an

`int *`, it reads off 32 ones and zeros from that position and converts them to a decimal number. The type is what told it to read 32 bits (4 bytes).

Now the same thing with a character:

```
char y = 'h';
char *b = &y;
printf("%c\n", *b);
```

This prints `h`. The `y` box is 8 bits (1 byte), four times smaller than the `x` box. When we dereference `b`, C goes to the address in the box and reads off one byte, because `b` is a `char *`.

And now the crazy bit. Nothing stops us from putting the address of `x` into `b`:

```
b = a;
printf("%c\n", *b);
```

Read that assignment carefully. It is `b = a`, not `b = &a`. It means: open the `a` box, take the number inside it and put that number in the `b` box. Both boxes now contain 30096. It turns out that that is an address. So both are aimed at the integer `x`.

But `b` is still a `char *`, and it will always be a `char *`. So when we dereference it, it reads one byte from address 30096: the first byte of that four-byte integer. The value 5 is stored as 4 bytes, and the first of these come out as the character with code 5, which is not a printable character, so what appears on the screen is junk. The compiler will warn you about the assignment (“assignment to `char *` from incompatible pointer type `int *`”) but it will let you do it.

If you genuinely mean it, you say so with a cast:

```
int *vPtr0;
char *vPtr2;

vPtr2 = (char *) vPtr0;
```

The cast silences the warning. It does not change the address. It changes how much gets read when you open the box, and how far `++` moves you.

While we are on the size of things, here is a fact worth knowing. How big is a pointer box? A pointer box always contains an address and never anything else, so every pointer box is the same size. On a 64-bit machine that is 64 bits (8 bytes). An `int *` is 8 bytes, and a `char *` is 8 bytes, even though the latter is aimed at something as small as a single character.

This is also the one place where the slogan “arrays are pointers” breaks down. If you ask `sizeof` about an array, you get the size of the whole array, but if you ask it about a pointer, you get 8:

```
int a[4];
int *p = a;

printf("%zu %zu\n", sizeof(a), sizeof(p)); // prints: 16 8
```

So an array knows how big it is and a pointer does not. Everywhere else the two behave the same way.

The void pointer

There is one special pointer type that is worth meeting now, even though we will not do much with it until later: the void pointer, written `void *`.

A `void *` points at anything. It can hold the address of a character, of an integer, of a structure, of anything at all. That makes it the generic pointer, and it is the one case where you do not need a cast: any pointer can be assigned to a `void *` without complaint.

```
int x = 5;
int *intPtr = &x;
void *voidPtr = intPtr;
```

void pointers are powerful precisely because they will carry any address. The one thing they cannot do is dereference. You cannot say “go to the box and give me the value”, because a `void *` does not know what kind of box it is aimed at. Is it a character, so read 1 byte? An `int`, so read 4? A double, so read 8? It has no idea, and neither does the compiler: `*voidPtr` is an error, not a warning.

The only way to get at the value is to tell C what is there, by casting back to a real pointer type first:

```
printf("%d\n", *(int *) voidPtr);
```

Reserving memory while the program runs

Let's switch topics. There is a machine learning site called Kaggle which hosts competitions: you practise your skills, and if you win you often get a substantial amount of money. The first challenge on the site is a toy problem, a little bit horrific and a little bit sad, called "Titanic: Machine Learning from Disaster". They give you a real dataset, the passenger manifest from the ship that sank.

	A	B	C	D	E	F	G	H	I	J	K	L	
1	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
2	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25		S	
3	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C	
4	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925		S	
5	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0		113803	53.1C123	S	
6	5	0	3	Allen, Mr. William Henry	male	35	0	0		373450	8.05	S	
7	6	0	3	Moran, Mr. James	male		0	0		330877	8.4583	Q	
8	7	0	1	McCarthy, Mr. Timothy J	male	54	0	0		17463	51.8625	E46	S
9	8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1		349909	21.075	S	
10	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2		347742	11.1333	S	
11	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14	1	0		237736	30.0708	C	
12	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549		16.7G6	S	
13	12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0		113783	26.55C103	S	
14	13	0	3	Saunders, Mr. William Henry	male	20	0	0	A/5. 2151		8.05	S	
15	14	0	3	Andersson, Mr. Anders Johan	male	39	1	5		347082	31.275	S	
16	15	0	3	Vestrom, Miss. Hulda Amanda Adolfina	female	14	0	0		350406	7.8542	S	
17	16	1	2	Hewlett, Mrs. (Mary D Kingcome)	female	55	0	0		248706	16	S	
18	17	0	3	Rice, Master. Eugene	male	2	4	1		382652	29.125	Q	
19	18	1	2	Williams, Mr. Charles Eugene	male		0	0		244373	13	S	
20	19	0	3	Vander Planke, Mrs. Julius (Emelia Maria Vandemoortele)	female	31	1	0		345763	18	S	
21	20	1	3	Massey, Mrs. Fatima	female		0	0		2649	7.225	C	
22	21	0	2	Fynney, Mr. Joseph J	male	35	0	0		239865	26	S	
23	22	1	2	Beesley, Mr. Lawrence	male	34	0	0		248698	13D56	S	
24	23	1	3	McGowan, Miss. Anna "Annie"	female	15	0	0		330923	8.0292	Q	
25	24	1	1	Sloper, Mr. William Thompson	male	28	0	0		113788	35.5A6	S	
26	25	0	3	Palsson, Miss. Torborg Danira	female	8	3	1		349909	21.075	S	
27	26	1	3	Asplund, Mrs. Carl Oscar (Selma Augusta Emilia Johanna)	female	38	1	5		347077	31.3875	S	
28	27	0	3	Emir, Mr. Farred Chehab	male		0	0		2631	7.225	C	
29	28	0	1	Fortune, Mr. Charles Alexander	male	19	3	2		19950	263C23 C25	S	
30	29	1	3	O'Dwyer, Miss. Ellen "Nellie"	female		0	0		330959	7.8792	Q	

Figure 4: Part of the Titanic passenger data. Each row is one passenger, and the last column records where they embarked: Southampton, Cherbourg or Queenstown. Data from the [Kaggle Titanic competition](#).

Each row holds a passenger's name, whether they survived, their sex, their age, their ticket number, what they paid, their cabin and the port where they got on. The task is to look at these passengers and then predict, for passengers who are not in this list, whether they survived. You can already see some of the pattern in the data: the men in this extract died and the women survived.

I only want you to think through this program (not actually write it). You are given a list of about a thousand passengers, so you could declare an array of a thousand and get on with it. But in real life, if you are running a shipping company, you do not know beforehand how many passengers will be on the boat. If you are a bank, you do not know how many customers you have (you probably hope the number keeps going up). So you don't want to fix your array sizes beforehand.

Everything we have written up to now has needed the size in advance. If you wanted a hundred things, you wrote `int a[100];` and the compiler set aside 400 bytes. What we want now is a way to say, in the middle of a running program, "reserve me 200 int-sized blocks", and then be able to use them.

The malloc command

The command for this is `malloc`, short for memory allocation. It lives in the standard library, so you need

```
#include <stdlib.h>
```

at the top of your program. A typical call looks like this:

```
newPtr = malloc(numberOfElements * sizeof(int));
```

There are three separate ideas in that one line.

The argument is a number of bytes. Not a number of elements, not a type: a plain count of bytes. `malloc(20)` asks for twenty bytes. That is why `sizeof` shows up here: `sizeof(int)` tells you how many bytes one `int` takes on this machine, so `numberOfElements * sizeof(int)` is how many bytes you need for that many of them. You could write `malloc(100 * 4)` if you knew `ints` were 4 bytes, but `sizeof` means you do not have to know and your program stays correct on a machine where they are not.

What `malloc` then does is find a chunk of memory that big, mark it as yours, and promise not to hand it to anyone else until you say so. The bytes are contiguous, one after another, exactly like the boxes of an array.

What does `malloc` give back? It gives back an address. Specifically, the address of the first byte of the block it just reserved. That is the whole of it. It does not return the memory, it does not return a special “block” object, it returns one number: where the block starts. That is why the result of `malloc` always goes into a pointer. A pointer is a box with an address in it, and an address is exactly what `malloc` hands you.

That address is the only handle you will ever get on that block. Lose it and the memory is gone: still reserved, but with no way to reach it.

Notice also what `malloc` does *not* know. It has no idea whether you are going to put characters, integers or anything else in there. You asked for bytes and it gave you bytes. So the address it hands back is a `void *`, the generic pointer from earlier in this lecture. That is why you can assign the result straight to a `char *` or an `int *` without a cast: assigning a `void *` to any pointer type is allowed.

One more thing can happen: the request can fail. If there is not enough memory, `malloc` does not crash, it returns `NULL`. A careful program checks:

```
newPtr = malloc(numberOfElements * sizeof(int));
```

```
if (newPtr == NULL)
{
    printf("Out of memory\n");
    return 1;
}
```

Dereferencing a NULL pointer is one of the reliable ways to crash a C program, so this is the check that stops a shortage of memory turning into a mysterious failure later.

Finally, and this is important: when you are finished with the block, you have to give it back. You tell C that it is allowed to use that memory for something else, with `free`:

```
free(newPtr);
```

You pass it the pointer that is holding the address of the block. If you do not do this, you get what is called a memory leak: every run of your program blocks off a chunk of memory and then never releases it, so the memory stays marked as unavailable. Keep doing that and the machine runs out. It is a very easy way to write something that behaves like a virus, so do not.

The pattern to remember is simply: `malloc`, use the memory, `free`.

Reserving memory for passengers

Here is a small program for a shipping company. It asks how many passengers are on the ship, reserves exactly that much memory, then reads in the port each passenger embarked from: C for Cherbourg, S for Southampton or Q for Queenstown.

```
/*
 * Dynamic assignment of arrays.
 */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    // Variables
    int i;
    char port;
    int noPassengers;
    char *portsOfEmbarkPtr; // starting point for a dynamic array

    // Determine size of dynamic array
    printf("How many passengers? ");
    scanf("%d", &noPassengers);

    // Allocate memory
    portsOfEmbarkPtr = malloc(noPassengers * sizeof(char));

    // Read embarkation ports
    for (i = 0; i < noPassengers; i++)
    {
        printf("Port: ");
```

```

        scanf(" %c", &port);
        portsOfEmbarkPtr[i] = port;
    }

    // Print embarkation ports
    for (i = 0; i < noPassengers; i++)
        printf("%c ", *(portsOfEmbarkPtr + i));

    free(portsOfEmbarkPtr);

    return 0;
}

```

Running it looks like this:

```

How many passengers? 5
Port: C
Port: S
Port: Q
Port: Q
Port: Q
C S Q Q Q

```

Walk through the interesting lines. `portsOfEmbarkPtr` is declared as a `char *`, so at that point it is just a pointer aimed at nothing in particular. The `malloc` line reserves `noPassengers` characters worth of memory, which for five passengers is five bytes, since `sizeof(char)` is 1. What comes back is the address of the first of those five bytes, and it goes into `portsOfEmbarkPtr`.

From then on the block behaves exactly like an array, because it is exactly like an array: contiguous boxes of a known size, with the address of the first one in a pointer. So `portsOfEmbarkPtr[i]` works, and it means what it always means: move on `i` character-sized steps from the address in the box, and use whatever is there. The printing loop deliberately uses the other notation, `*(portsOfEmbarkPtr + i)`, to make the point that the two are identical. Swap them around and the program does the same thing.

The `scanf(" %c", &port)` has the little space before the `%c` that trips everyone up; it is there to skip the newline left behind by the previous input.

And at the end, `free(portsOfEmbarkPtr)` hands the five bytes back.

So the program created its array at runtime, sized by something the user typed, and reached into it using nothing but the knowledge that arrays and pointers are friends. In a real program you would add the `NULL` check after the `malloc`, which is left as one of the exercises below.

Exercises

1. Given `int a[4] = {99, 1, 3, 26}` and `int *p = a`, work out what each of `*(p + 1)`, `p[3]` and `*(a + 0)` gives, then write a program to check your answers.
2. Print `&a[0]`, `&a[1]` and `&a[2]` for the array above using `%p`. Convert the three addresses from hexadecimal to decimal and confirm that the gaps are `sizeof(int)`. (If you don't know hexadecimal, that is okay: ask Google to do the conversion for you.) Now do the same for a `char c[4]` and explain why the gaps are different.
3. Rewrite the printing loop of the passenger program so that instead of an index it keeps a `char *` that starts at `portsOfEmbarkPtr` and is incremented each time round. Use a pointer comparison rather than a counter to decide when to stop.
4. Print `sizeof(a)` and `sizeof(p)` for the array and pointer of exercise 1. Explain the two numbers you get. Which of them would change if the array had eight elements?
5. Take the passenger program and add the `NULL` check after the `malloc`, printing a sensible message and returning 1 if the allocation failed. Then try to make the check actually trigger by asking for an absurd number of passengers.
6. Write a program that asks the user how many numbers they want to enter, reserves exactly that many `ints` with `malloc`, reads them in, prints their average and frees the memory. Use subscript notation for reading and pointer/offset notation for the loop that adds them up.
7. In the passenger program, delete the `free` and run it. Nothing visibly goes wrong. Write a paragraph explaining what has nevertheless happened, and why it would matter in a program that ran for hours rather than seconds.